INSURANCE CLAIM MANAGEMENT

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Use Case Report

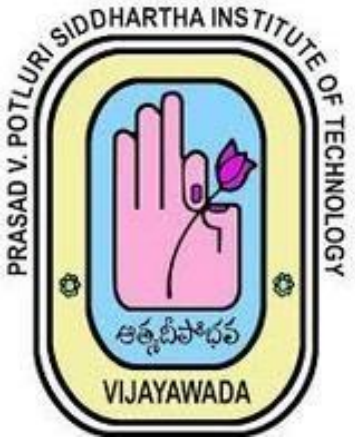
submitted by

ANNURI NIKOLA WESLY

23501A0511

Under the guidance of

Mr. A. Prashant, Asst. Prof.



Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

2026-27

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the Use Case report entitled **"Insurance Claim Management"** that is being submitted by **Annuri Nikola Wesly (23501A0511)**, as part of Assignment-2 for the **Blockchain Technology** course during the academic year **2026-27**. The report is prepared on the topic of insurance claim workflow management using blockchain records, smart-contract rules, and digital surveyor verification.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS
ASSIGNMENT-2 ____ /5

INDEX

S. No.	Chapter	Page No. (approx.)
1	Introduction	1
2	Background and Problem Statement	3
3	Blockchain and Ethereum Basics	5
4	Use Case Overview	7
5	System Architecture and Workflow	10
6	Implementation Design	14
7	Benefits	19
8	Challenges and Security Considerations	20
9	Conclusion and Future Enhancements	22
10	SDGs Addressed	23
11	References	24
12	Appendix A – Demonstration Checklist	26

Note: Page numbers are approximate and may change slightly depending on Word rendering.

1. INTRODUCTION

1.1 Overview of Blockchain Technology

Blockchain is a distributed ledger in which transactions are grouped into blocks and linked using cryptographic techniques. Copies of the ledger are maintained by participating nodes, which validate updates according to the network's protocol. This structure can make recorded events difficult to change without detection and allows authorized participants to inspect the transaction history.

Smart-contract platforms extend blockchain beyond simple value transfers. A smart contract is a program deployed on a blockchain that checks predefined rules and updates on-chain state when valid transactions are submitted. In an insurance workflow, such rules can record claim status changes, restrict actions to approved roles, and preserve a traceable history of important decisions.

1.2 Relevance to Insurance Claim Management

Insurance claim processing commonly involves a policyholder, an insurance company, a surveyor or assessor, supporting documents, a claim decision, and a settlement. Information may pass between different departments and systems. Delays, duplicate submissions, inconsistent records, missing documents, and uncertainty about the current status can reduce trust in the process.

A blockchain-based approach can provide a shared, tamper-evident log of key claim events. It does not prove that every uploaded document is truthful, nor can it independently inspect physical damage. Therefore, the proposed system combines digital evidence submission with a designated surveyor's verification and stores sensitive documents outside the public ledger.

1.3 Proposed Solution

The proposed system, referred to as EcoChain in the project concept, is a web-based insurance claim management prototype. A policyholder submits a claim and uploads supporting evidence through the application. The system records a claim identifier, status, timestamp, and evidence hash or reference. An authorized surveyor reviews the submitted evidence and records a verification result with a brief reason. An insurer or claims administrator then reviews the verified claim and records approval or rejection. For an approved claim, the settlement status and transaction reference are recorded.

The digital survey step means that evidence and verification are handled through the application. It does not imply that physical inspection is unnecessary in every real insurance case. When remote evidence is insufficient, the insurer may still require an in-person inspection under its normal procedures.

1.4 Objectives

- Maintain a traceable history of claim submission, verification, decision, and settlement.
- Reduce ambiguity about claim status and responsibility for each workflow step.
- Support role-based actions for policyholders, surveyors, and insurer administrators.
- Use evidence hashes or protected references to help detect later file changes.
- Demonstrate a blockchain consensus approach suitable for a controlled academic network.
- Keep sensitive personal information and full evidence files off-chain.

1.5 Organization of the Report

Chapter 2 discusses the background and problem statement. Chapter 3 introduces blockchain concepts and the proposed Proof of Authority (PoA) network. Chapter 4 defines actors, scope, and requirements. Chapter 5 presents the architecture and workflow. Chapter 6 describes the proposed implementation, data model, smart-

contract logic, and testing plan. Chapters 7–10 discuss benefits, risks, conclusion, future enhancements, and relevant Sustainable Development Goals.

2. BACKGROUND AND PROBLEM STATEMENT

2.1 Existing Claim-Processing Challenges

- Documents may be submitted through different channels, making it difficult to maintain one consistent claim history.
- Policyholders may have limited visibility into whether a claim is submitted, under review, verified, approved, rejected, or settled.
- Repeated data entry across departments can introduce mistakes or inconsistent records.
- Disputes may arise over when evidence was submitted or which version of a document was reviewed.
- Manual follow-ups can increase processing time and administrative effort.
- Unauthorized changes in a centralized database may be difficult for external parties to independently audit.

2.2 Why a Shared Audit Trail Helps

A blockchain ledger can record selected events such as ClaimSubmitted, SurveyorVerified, ClaimApproved, ClaimRejected, and ClaimSettled. Each event can be associated with a claim identifier, transaction hash, timestamp, and role address. Authorized users can compare the event history with the application's displayed status. This improves traceability but does not guarantee that the original evidence or human decision was correct.

2.3 Digital Surveyor Verification

In the proposed workflow, the surveyor signs in through an authorized account and reviews the policy details and evidence submitted by the policyholder. The surveyor can record one of the configured outcomes, such as Verified, More Information Required, or Not Verified, along with a reason code. The system records the result and the evidence hash or reference reviewed. This is digital verification of submitted evidence and workflow records.

A single designated surveyor is used in the basic prototype to keep the workflow understandable. The system does not automatically guarantee that the surveyor is honest. Role approval, audit logs, evidence cross-checks, random audits by the insurer, and accountability for the surveyor account are possible mitigations. A second-surveyor approval is not required in the minimal design.

2.4 Problem Statement

The problem addressed is the lack of a simple, auditable and role-controlled workflow for managing insurance claims from submission to settlement. The proposed system aims to improve the visibility and integrity of claim-event records while retaining human judgement for evidence assessment and claim decisions.

2.5 Proposed Approach and Boundaries

Blockchain is used for the minimum shared record and workflow state, while a conventional database or protected file store holds detailed policy information, photographs, reports, and personal data. The insurer remains responsible for policy interpretation, fraud investigation, regulatory compliance, and final claim handling. The prototype supports record traceability; it is not a replacement for an insurer's core system or legal process.

3. BLOCKCHAIN AND ETHEREUM BASICS

3.1 Distributed Ledger

A distributed ledger stores replicated records across network participants. Each participant validates transactions according to the selected protocol. This makes it possible for approved participants to share a common event history rather than relying entirely on a single mutable database.

3.2 Smart Contracts

Smart contracts are programs deployed at blockchain addresses. They can enforce checks such as whether the caller has the surveyor role, whether a claim is in the correct state, and whether a claim has already been settled. A contract cannot itself determine whether a photograph shows genuine damage; this requires human review or a separately designed trusted evidence process.

3.3 Proposed Consensus: Proof of Authority

For a controlled academic demonstration, the project may use a Proof of Authority (PoA) network. In PoA, a limited set of approved validator nodes participate in proposing or validating blocks according to the network's rules. Validators are selected based on an established identity or authorization process rather than open mining or token-staking competition.

In this use case, the insurer's controlled consortium or test environment can operate approved validator nodes. The policyholder and surveyor do not automatically become validators simply because they submit or verify a claim. Their application roles determine which contract actions they can call; validator nodes are responsible for block production or validation at the network level.

PoA is suitable for a demonstration where participants are known and governance is controlled. It is not equivalent to a fully public, permissionless network, and it depends on trust in validator selection and operation. The exact PoA mechanism depends on the chosen compatible blockchain client or network configuration.

3.4 Wallets, Addresses, and Transactions

A wallet or managed signing account authorizes a transaction using cryptographic keys. The address identifies the transaction caller but does not by itself prove the person's real-world identity. In an insurance system, the application should bind an approved account to a verified user role and protect private keys or signing credentials.

3.5 On-Chain and Off-Chain Data

Data placement	Examples	Reason
On-chain	Claim ID, status, timestamps, evidence hash, decision event, settlement reference	Shared auditability and integrity checks
Off-chain	Policyholder name, address, policy documents, damage photographs, detailed survey report	Privacy, file size, access control, and confidentiality
Frontend/backend	Forms, authentication, notifications, search, protected file access	Usable workflow and integration with existing systems

3.6 Consensus Versus Business Approval

Consensus and claim approval are different processes. Consensus determines which transactions and blocks are accepted by the blockchain network. Claim approval is a business decision made by an authorized insurer role after considering policy terms and surveyor findings. A block being validated does not mean that an insurance claim is genuine or approved.

4. USE CASE OVERVIEW

4.1 Scope

The prototype demonstrates the lifecycle of a claim from submission to a recorded decision and settlement status. It uses a simple role-based workflow and a blockchain ledger for audit events. It is designed for academic demonstration with sample data and a test network.

4.2 Main Actors

Actor / component	Responsibilities
Policyholder	Signs in, submits claim details, uploads evidence, tracks status, and responds to requests for more information.
Surveyor	Reviews submitted evidence digitally, records a verification outcome and reason, and identifies missing information.
Insurer administrator / claims officer	Reviews claim details and surveyor result, records approval or rejection, and initiates or records settlement.
Smart contract	Checks role permissions and valid state transitions; records claim events and selected references.
Frontend application	Provides forms, dashboards, status messages, and role-specific actions.
Backend and protected storage	Manages authentication, document access, metadata, notifications, and application integration.
PoA validator nodes	Validate and order blockchain transactions according to the configured network protocol.

4.3 Functional Requirements

ID	Requirement
FR-1	A policyholder can submit a claim with required fields and evidence references.
FR-2	The system generates or accepts a unique claim identifier.
FR-3	The application records an evidence hash or protected file reference.
FR-4	Only an authorized surveyor can record the survey verification outcome.
FR-5	The surveyor can mark evidence as verified, not verified, or requiring more information.
FR-6	Only an authorized insurer role can approve or reject a verified claim.
FR-7	The system records the claim decision and its timestamp.
FR-8	A settled claim cannot be settled a second time through the same workflow.
FR-9	Users can view claim status and relevant transaction history according to permissions.
FR-10	Failed or rejected transactions must be displayed as failures, not successes.

4.4 Non-Functional Requirements

Quality attribute	Requirement
Security	Role checks, secure authentication, input validation, protected file access, and key management.
Privacy	Keep personal data and full claim documents off-chain.
Reliability	Test valid and invalid transitions and handle failed transactions.
Auditability	Retain event identifiers, timestamps, evidence hashes, and transaction references.
Usability	Use clear claim statuses and explanations for missing information.
Maintainability	Separate UI, backend, storage, and smart-contract responsibilities.

4.5 Illustrative Scenario

1. The policyholder logs in and submits a motor-damage claim with a policy number, incident description, date, and photographs.
2. The application stores files in protected off-chain storage and calculates a hash for each relevant evidence file.
3. The claim record and evidence hash/reference are submitted to the blockchain, and the status becomes Submitted.
4. The authorized surveyor reviews the evidence digitally and records Verified, More Information Required, or Not Verified.
5. If more information is required, the policyholder uploads additional evidence and the workflow returns to survey review.
6. The insurer's claims officer checks the policy and surveyor result, then records Approved or Rejected with a reason.
7. For an approved claim, settlement is processed according to the prototype's defined method, and the settlement status and reference are recorded.
8. The policyholder and authorized staff can inspect the claim timeline and verify the recorded events.

4.6 Out of Scope

- Automatic determination of whether a real-world accident or damage event occurred.
- Guaranteed detection of forged photographs or dishonest surveyors.
- Replacing physical inspection where the insurer or law requires it.
- Automated legal arbitration or interpretation of all policy clauses.
- Storing complete personal documents or photographs on a public blockchain.
- Real-money insurance payouts in an unreviewed academic prototype.

5. SYSTEM ARCHITECTURE AND WORKFLOW

5.1 Architecture Layers

Layer	Proposed components	Purpose
User layer	Policyholder, surveyor, claims officer	Perform role-specific actions.
Presentation layer	React web interface	Forms, dashboards, evidence upload, claim timeline.
Application/API layer	Node.js/Express or equivalent backend	Authentication, validation, notifications, and service orchestration.
Evidence layer	Protected database and file storage	Store policy metadata, photographs, reports, and access permissions.
Blockchain interface	Wallet/provider and contract client	Submit transactions and read contract state/events.
Smart-contract layer	Solidity contract on an EVM-compatible PoA network	Role checks, state transitions, claim event records.
Network layer	Approved PoA validator nodes	Validate and record transactions into blocks.

5.2 Architecture Diagram

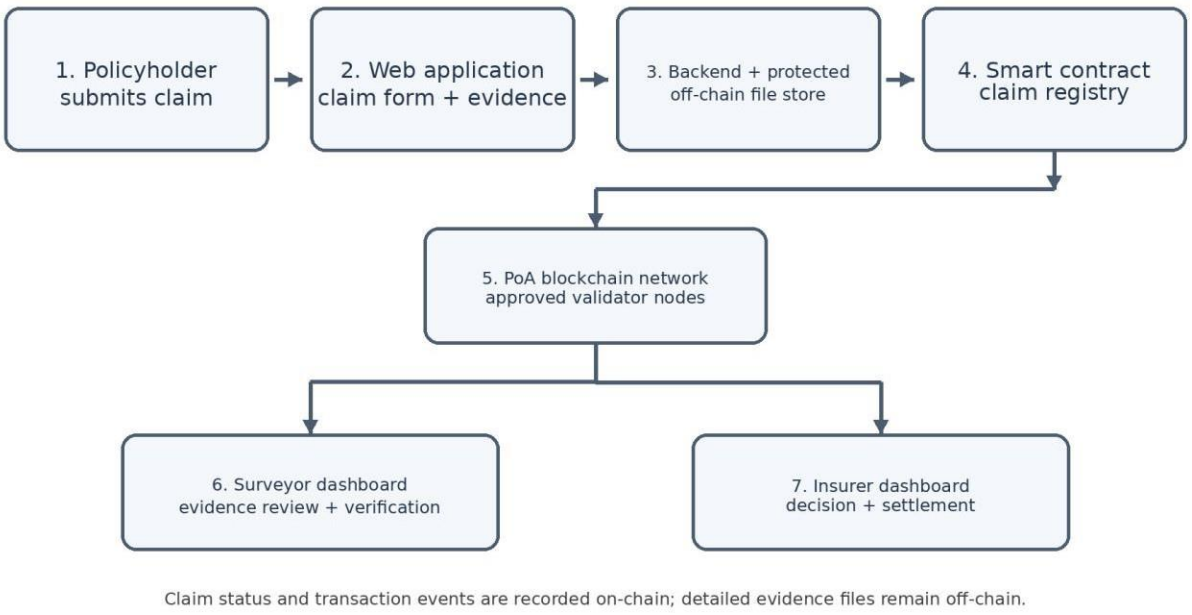


Figure 5.1: Proposed architecture for blockchain-based insurance claim management.

5.3 Claim Lifecycle and State Transitions

Current state	Authorized action	Next state
Not Created	Policyholder submits a valid claim	Submitted
Submitted	Surveyor starts review	Under Review
Under Review	Surveyor confirms evidence is sufficient	Verified
Under Review	Surveyor requests further evidence	More Information Required
More Information Required	Policyholder submits requested evidence	Submitted / Under Review
Verified	Claims officer approves claim	Approved
Verified	Claims officer rejects claim with reason	Rejected
Approved	Settlement is processed and recorded	Settled

The contract should reject invalid state changes, such as settlement before approval, verification by an unauthorized account, or a second settlement of the same claim.

5.4 Workflow Diagram

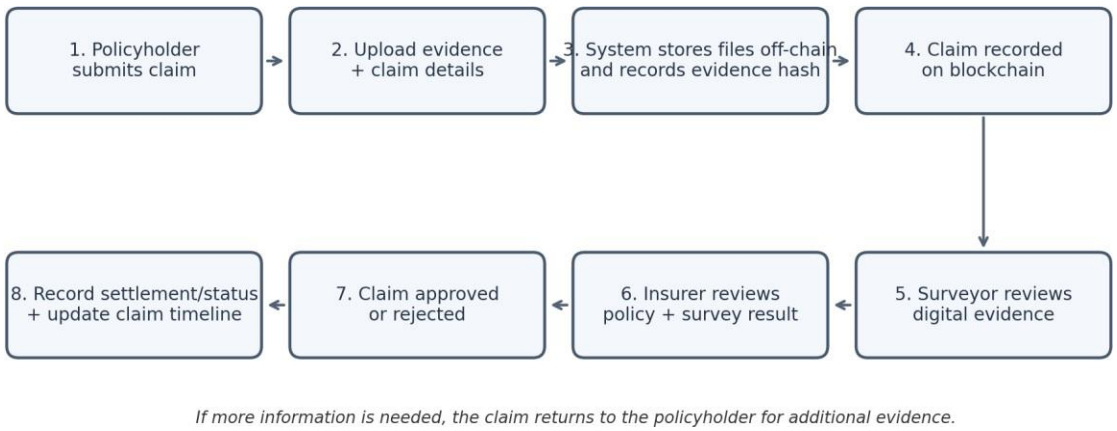


Figure 5.2: Proposed claim workflow. Digital review handles submitted evidence; physical inspection may still be required in real cases.

5.5 Role of the Surveyor

The surveyor is an authorized reviewer, not a blockchain validator. The surveyor examines submitted evidence, checks whether details are consistent with the claim, and records a signed verification outcome. The system can preserve which authorized account recorded the outcome and when. A dishonest or compromised surveyor account remains a risk; the ledger makes the action traceable but does not make the judgement automatically true.

5.6 Role of the Administrator / Claims Officer

The claims officer is responsible for the business decision after considering policy coverage, exclusions, claim limits, and surveyor findings. The officer records approval or rejection with a reason. The administrator does

not need to manually validate every blockchain transaction; network validators perform consensus at the blockchain layer.

5.7 Data Placement and Privacy

Only minimal records should be written to the chain: claim ID, status, event timestamp, evidence hash or protected reference, role address, and settlement reference. Names, addresses, policy documents, detailed incident descriptions, photographs, and sensitive details should remain off-chain with encryption and access controls. A hash can help detect a changed file, but does not establish that the original file is authentic.

6. IMPLEMENTATION DESIGN

6.1 Suggested Technology Stack

Component	Suggested technology	Purpose
Frontend	React.js, HTML, CSS, Bootstrap	Role-based dashboards and forms.
Backend	Node.js and Express.js	APIs, access control, validation, and workflow orchestration.
Database	MongoDB or MySQL	Policy/claim metadata and off-chain application records.
File storage	Protected local storage or cloud object storage	Evidence documents and photographs.
Smart contract	Solidity 0.8.x	Claim registry, role checks, state transitions, and events.
Blockchain network	EVM-compatible private/consortium PoA network or local development network	Controlled consensus and event recording.
Development tools	Remix or Hardhat; compatible wallet/provider	Compile, test, deploy, and interact with contract.

6.2 Claim Data Model

A claim record can include a unique claim ID, policy reference, claimant account, incident category, incident date, evidence hash/reference, assigned surveyor, verification result, decision status, timestamps, and settlement reference. Sensitive fields should be held off-chain. On-chain storage should be minimized because writes can cost resources and public records may be difficult to erase.

Field	Example / meaning	Storage recommendation
claimId	Unique identifier such as CLM-1001	On-chain and off-chain index
claimantAddress	Authorized account submitting the claim	On-chain only if necessary; otherwise protected mapping
policyReference	Internal policy identifier	Prefer off-chain; use opaque reference on-chain
evidenceHash	Cryptographic hash of uploaded file	On-chain
evidenceFile	Photo, invoice, report	Off-chain protected storage
surveyorOutcome	Verified / More Information / Not Verified	On-chain event/status and off-chain detail
decision	Approved / Rejected	On-chain event/status
settlementReference	Reference to payment or settlement record	On-chain reference; sensitive details protected

6.3 Smart-Contract Responsibilities

- Maintain claim identifiers and current workflow status.
- Restrict survey verification functions to approved surveyor accounts.
- Restrict approval, rejection, and settlement-recording functions to authorized insurer accounts.
- Reject invalid state transitions and duplicate settlement actions.
- Emit events for submission, verification, decision, and settlement.

- Store only hashes or opaque references rather than confidential evidence files.

6.4 Example Contract Logic (Pseudocode)

```
enum ClaimStatus {
    Submitted,
    UnderReview,
    MoreInformationRequired,
    Verified,
    Approved,
    Rejected,
    Settled
}

struct Claim {
    address claimant;
    bytes32 evidenceHash;
    ClaimStatus status;
    address surveyor;
    uint256 submittedAt;
}

function submitClaim(bytes32 evidenceHash) external {
    // Validate required fields, create a unique claim, emit event.
}

function recordSurveyResult(uint256 claimId, uint8 result)
    external onlyAuthorizedSurveyor {
    // Validate state, record outcome, emit SurveyResultRecorded.
}

function recordDecision(uint256 claimId, bool approved)
    external onlyClaimsOfficer {
    // Require a verified claim; record approval or rejection.
}

function recordSettlement(uint256 claimId, bytes32 settlementRef)
    external onlyClaimsOfficer {
    // Require Approved state and prevent duplicate settlement.
}
```

Listing 6.1: Illustrative pseudocode only; it is not a complete, compiled Solidity contract.

6.5 Role-Based Access Control

The contract or trusted application authorization layer should maintain separate permissions for policyholders, surveyors, and claims officers. Registration alone must not automatically grant privileged roles. An administrator should approve role assignments, and role changes should be logged. In production, key rotation, revocation, and compromised-account procedures must be defined.

6.6 Digital Evidence Verification

9. The policyholder uploads a document or photograph through an authenticated form.
10. The application validates file type and size and stores the file in protected off-chain storage.
11. The application calculates a cryptographic hash of the exact file bytes.
12. The claim record stores the hash and a protected reference to the file.

13. The surveyor opens the file through an authorized link and reviews the evidence.

14. If a later copy is presented, its hash can be recalculated and compared with the recorded hash to detect whether the bytes changed.
15. The surveyor records the review result and reason; the blockchain records the event.

A matching hash only indicates that the compared file bytes match. It does not prove when or where a photograph was taken, or whether the content is genuine. Those questions require contextual checks and, where necessary, physical inspection.

6.7 PoA Validation and Transaction Flow

16. A user performs an action in the web application and confirms the transaction using the configured signing method.
17. The application submits the signed transaction to the configured blockchain node or RPC endpoint.
18. Approved PoA validators process the transaction according to the network's consensus rules.
19. If accepted, the transaction is included in a block and the contract state/events are updated.
20. The application waits for the required confirmation policy before showing the action as recorded.
21. The UI displays the transaction hash and current claim status; failed transactions are shown as failed.

6.8 Security Controls

- Validate all user inputs and uploaded files on the server side.
- Use explicit role checks for each privileged contract function.
- Prevent invalid state transitions and duplicate approval or settlement.
- Protect private keys, signing credentials, and session tokens.
- Do not expose evidence files through public URLs without access controls.
- Use HTTPS, least-privilege access, audit logs, backups, and secure secret configuration.
- Test failure paths and review the smart contract before any real-world use.

6.9 Testing Plan

Test case	Expected result
Submit claim with valid data	Claim ID is created and status becomes Submitted.
Submit missing required fields	Application rejects the request with a useful message.
Unauthorized user records survey result	Action is rejected.
Authorized surveyor records verification	Outcome and event are recorded.
Surveyor requests more information	Status changes and claimant can upload additional evidence.
Claims officer approves unverified claim	Contract/workflow rejects the invalid transition.
Claims officer rejects a verified claim	Rejection status and reason are recorded.
Attempt settlement before approval	Action is rejected.
Attempt settlement twice	Second settlement is rejected.
Evidence file changes after upload	Recalculated hash does not match the recorded hash.
Blockchain transaction fails	UI reports failure and does not show success.

6.10 Deployment and Implementation Status

A development lifecycle should include local compilation, unit testing, deployment to a controlled test network, role and workflow testing, and recording of the contract address and transaction evidence. If the application is only a design or frontend prototype, the report and presentation must say so clearly. Do not

claim that real blockchain consensus, digital signatures, or settlement automation is operational unless it has been implemented and demonstrated.

Suggested evidence to include after implementation: compiler version, network name, chain ID, deployed contract address, test output, sample claim ID, transaction hashes, and screenshots of each workflow state. This report specifies a proposed design and does not assert that these deployment steps have already been completed.

7. BENEFITS

7.1 Traceable Claim History

A sequence of blockchain events can show when a claim was submitted, reviewed, verified, approved, rejected, or marked settled. This makes the recorded workflow easier to audit.

7.2 Improved Status Transparency

A policyholder can view a clear status and the next required action rather than repeatedly contacting the insurer for updates. Role-specific views can reduce confusion while preserving privacy.

7.3 Evidence Integrity Checks

A cryptographic hash can help identify whether a stored file has changed since its reference was recorded. This supports integrity checking but does not establish the truth of the evidence.

7.4 Role Accountability

Role-restricted actions and event logs can help identify which authorized account recorded a verification or decision and when it occurred. This supports accountability but does not prevent collusion or compromised credentials by itself.

7.5 Reduced Record Reconciliation

A shared event history may reduce disagreement between systems about the sequence of claim actions. The benefit depends on integration quality and on participants recording events correctly.

7.6 Extensibility

The design can later be extended with policy verification, notifications, document versioning, configurable claim categories, analytics, integrations with insurer systems, and stronger review procedures.

8. CHALLENGES AND SECURITY CONSIDERATIONS

8.1 Fraudulent or Misleading Evidence

A blockchain can preserve the hash of a file but cannot guarantee that a photograph, invoice, or report depicts a genuine event. Mitigations include file metadata checks, consistency checks, contextual evidence, insurer investigation, and physical inspection when needed.

8.2 Surveyor Misconduct or Account Compromise

The single-surveyor prototype keeps the workflow simple, but a dishonest surveyor can still submit an incorrect result. Mitigations include verified role assignment, individual accountability, recorded reasons, random audits, monitoring unusual patterns, and revoking compromised accounts. A second surveyor is an optional governance enhancement, not a requirement of the basic workflow.

8.3 Privacy and Confidentiality

Insurance claims can contain sensitive personal, financial, property, or health information. Public blockchains are inappropriate for storing raw confidential documents. Store full files off-chain with encryption, access controls, retention rules, and secure backups. Avoid personally identifying information in public event data.

8.4 Smart-Contract Vulnerabilities

Incorrect access control, state-transition bugs, reentrancy in payment-related functions, unsafe external calls, and poor key management can undermine the system. Use established libraries, automated tests, static analysis, and independent security review. An academic prototype should use sample data and test funds only.

8.5 PoA Governance and Availability

PoA relies on approved validators and clear governance. If too many validators are unavailable or collude, network availability or trust can be affected. The project should document who operates validators, how validators are added or removed, how keys are protected, and what happens during outages.

8.6 Physical Inspection Limitations

Digital evidence is suitable for some straightforward claims, but it may be inadequate for complex damage, disputed incidents, or claims requiring professional examination. The system should allow a claim to be marked for additional information or physical inspection rather than automatically approving it.

8.7 Settlement and Legal Constraints

Recording an approved or settled status on-chain does not itself transfer money unless a payment mechanism has been separately implemented. Real insurance settlements must follow policy terms, payment controls, regulatory requirements, and organizational procedures.

8.8 Risk and Mitigation Summary

Risk	Mitigation
False evidence	Evidence cross-checks, metadata review, investigation, and physical inspection when needed.
Dishonest surveyor	Verified role, recorded reasons, audit sampling, monitoring, and role revocation.
Unauthorized actions	Role-based permissions, secure authentication, and contract-level access checks.
Privacy leakage	Keep full documents off-chain; encrypt files and apply

	access controls.
Invalid state changes	Define a state machine and test every permitted and forbidden transition.
PoA validator failure	Document validator governance, monitoring, backups, and recovery procedures.
False success shown in UI	Wait for transaction confirmation and display pending/confirmed/failed states accurately.
Unreviewed deployment	Use a test network and obtain independent review before production use.

9. CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This report presents a proposed blockchain-based insurance claim management system focused on traceable claim events, digital surveyor verification, role-controlled decisions, and settlement-status recording. The policyholder submits evidence, the authorized surveyor reviews it digitally, and the insurer's claims officer makes the business decision. The blockchain records selected events and evidence references, while sensitive files remain in protected off-chain storage.

The proposed Proof of Authority network is intended for a controlled academic or consortium demonstration where validator nodes are approved. Validators validate blockchain transactions; surveyors verify claim evidence; and claims officers decide whether a claim should be approved. These are distinct responsibilities. Blockchain improves the audit trail but cannot guarantee that submitted evidence is truthful or that human decisions are correct.

9.2 Future Enhancements

- Add notifications for claim submission, missing information, decision, and settlement.
- Integrate a secure document viewer with access expiry and file-version tracking.
- Introduce configurable claim categories and policy rule checks.
- Add audit dashboards to flag repeated patterns or unusual surveyor outcomes.
- Integrate with a real insurer payment system only after legal, security, and operational review.
- Consider multi-party review for high-value or disputed claims if required by policy.
- Add role revocation, key rotation, disaster recovery, and validator governance tooling.
- Evaluate privacy-preserving approaches and permissioned deployment options.

9.3 Suggested Development Roadmap

22. Build the role-based frontend and off-chain claim/evidence storage.
23. Implement the claim state machine and role restrictions in the smart contract.
24. Configure the local or private PoA development environment and test validator behavior.
25. Connect the application to the contract and replace any mock status changes with actual transactions.
26. Run positive and negative tests for each role and state transition.
27. Demonstrate a sample claim from submission through decision and settlement status.
28. Document actual test evidence, limitations, and any features not implemented.

10. SUSTAINABLE DEVELOPMENT GOALS ADDRESSED

SDG 9: Industry, Innovation and Infrastructure

The project demonstrates a digital infrastructure concept that combines web applications, access control, evidence integrity checks, and distributed records. Its contribution depends on secure implementation and practical integration.

SDG 16: Peace, Justice and Strong Institutions

Traceable claim events and recorded decision reasons can support accountability and auditability. The system does not guarantee fairness; transparent procedures, appeal options, and proper governance remain important.

SDG 10: Reduced Inequalities

A clear online claim-status interface may help users understand the progress of their claims. Accessibility still depends on internet access, device availability, language, and support for users who cannot use digital channels.

SDG 12: Responsible Consumption and Production

Digital document handling may reduce some paper-based administration. Environmental impact depends on the selected network and infrastructure; the report does not claim that every blockchain system is automatically sustainable.

11. REFERENCES

29. Ethereum Foundation, “Ethereum Developer Documentation.” https://ethereum.org/en/developers/docs/
30. Solidity Team, “Solidity Documentation.” https://docs.soliditylang.org/
31. OpenZeppelin, “Contracts Documentation: Access Control and Security.” https://docs.openzeppelin.com/contracts/
32. Hardhat, “Hardhat Documentation.” https://hardhat.org/docs
33. ethers.js, “Documentation.” https://docs.ethers.org/
34. MetaMask, “Developer Documentation.” https://docs.metamask.io/
35. NIST, “Secure Hash Standard (SHS), FIPS PUB 180-4.” https://csrc.nist.gov/pubs/fips/180-4/upd1/final
36. World Wide Web Consortium (W3C), “Web Content Accessibility Guidelines (WCAG).” https://www.w3.org/WAI/standards-guidelines/wcag/
37. United Nations, “Sustainable Development Goals.” https://sdgs.un.org/goals

References are provided as starting points for the technologies and general concepts described in this report. Add any course-prescribed sources or project-specific sources required by the instructor.

12. APPENDIX A – DEMONSTRATION CHECKLIST

A.1 Sample Claim

Item	Sample value (for demonstration only)
Claim ID	CLM-1001
Claim type	Motor damage
Claimant	Sample policyholder account
Evidence	Two sample photographs and one sample invoice
Surveyor outcome	Verified / More Information Required / Not Verified
Claims officer decision	Approved or Rejected with reason
Final status	Settled only after settlement is actually recorded

A.2 Demonstration Checklist

- Show login or role selection for policyholder, surveyor, and claims officer.
- Submit a claim with sample data and upload evidence.
- Show that evidence files are stored off-chain and a hash/reference is recorded.
- Show the claim ID and Submitted status.
- Attempt a surveyor-only action with an unauthorized account and show rejection.
- Record a surveyor outcome using an authorized account.
- Demonstrate More Information Required and subsequent evidence upload if implemented.
- Record an approval or rejection through the authorized claims officer role.
- Attempt settlement before approval and show that it is rejected.
- Record settlement status once and confirm duplicate settlement is prevented.
- Display event history and transaction hashes or local-chain receipts.
- Clearly state which features are implemented, simulated, or planned.

A.3 Important Implementation Note

This report describes a proposed academic system. Do not represent mock transactions, screenshots, or diagrams as proof of live blockchain deployment. For a deployed demonstration, record the actual network, chain ID, contract address, transaction hashes, test results, and known limitations.

A.4 Suggested Project Structure

```
insurance-claim-management/
    contracts/
        InsuranceClaimRegistry.sol
    test/
        InsuranceClaimRegistry.test.js
    scripts/
        deploy.js
    backend/
        routes/
        controllers/
        services/
        models/
    frontend/
        src/
            components/
```

```
        pages/
        services/
uploads-or-storage-config/
hardhat.config.js
package.json
README.md
```

A.5. Project Files (Google Drive)

https://drive.google.com/drive/folders/1wNw3iOKgQigZatS-HnrH2zbuA487jheB?dmr=1&ec=wgc-drive-%5Bmodule%5D-goto

A.6. Github

https://github.com/Nikolawesly/Blockchain